AI-Augmented Software Engineering-A Hands-On Journey Across the SDLC pathway - Assignment 2

Specification Driven Team Standup Service

Target competency	Duration	Primary tool	Submission
Specification driven development	1 week	Copilot	Seven Markdown files in docs only

Purpose

Build a locally runnable Team Standup Service by starting from these specifications and using AI throughout the SDLC. The assignment assesses how well you clarify requirements, make evidence-based decisions, control coding-agent work and verify the result.

The application matters, but reviewers will assess the quality and consistency of the evidence in your seven Markdown files. Code is retained locally and is not submitted.

What you must do

Step	Action	Main evidence
1	Understand the product and resolve the nine open decisions.	01-planning.md
2	Investigate one material uncertainty for 30 to 60 minutes.	01-planning.md
3	Write an implementation-ready specification and test it with a fresh agent.	02-specification.md
4	Design the solution and divide delivery into at least six bounded tasks.	03-design.md and 04-build.md
5	Implement one task at a time using controlled agent context and Git branches.	04-build.md
6	Test the requirements, review findings and verify fixes.	05-testing.md and 06-review.md
7	Reflect on human decisions, rework and lessons.	07-reflection.md

What you submit

- Submit only the docs folder containing the seven required Markdown files and demo video link.
- Do not submit application code.
- Keep the local working repository and runnable environment until the program confirms that review is complete.

Where do you submit

- First, create an Assignment-2 folder in your respective GitLab repository.
- Inside Assignment-2, create a docs folder.
- Submit your assignment only in the docs folder.

Important Points

Keep Files in Sync

You write these files as you work, so later phases will change earlier decisions. When that happens:

- **Update the earlier file so it reflects** your final position, for example a changed data model goes back into 02-design.md.
- **Record what changed and why** in the phase where the change came up, such as Implementation Plan vs. Actual in 03-build.md or Findings in 05-review.md.
- **Never leave two files disagreeing** about the same decision, data model, or requirement status.

Confidentiality, Data & Licensing

Treat everything you submit as material that will be read outside your team.

- **No personal data.** Use synthetic test data only. No real names, emails, phone numbers, addresses, or account details in examples, test data, screenshots, or logs. Redact anything personal that **appears in a screenshot.**
- **No client or employer material.** No client code, client names, internal URLs, internal system names, proprietary data, or copied internal documentation.
- **No secrets.** No API keys, tokens, passwords, or connection strings in text, screenshots, or logs.
- **Properly licensed tools only.** Prefer open-source or free-to-use tools, libraries, and plugins. Never use a paid tool without a valid license for this use — no unlicensed, cracked, or expired-trial software.
- **No copied proprietary code.** If an AI suggestion appears to reproduce licensed or proprietary code, do not use it — ask for an original implementation instead

1. Product definition

A typical day

A delivery team of about 20 people uses written standup. Each member records a short task list and optional blockers for an explicit team date. By mid-morning, members record today's tasks and may correct the previous working day. On Monday, that means Friday and Monday. The team lead checks missing submissions and blockers and exports one day as Markdown. Some members travel, so one configured team timezone determines the team date. Support staff need health and failure information without exposing standup text.

Fixed scope

- One team per deployment. The roster is loaded from configuration at startup and contains 1 to 50 members. There is no roster-management UI or API.
- One entry exists per member per team date and contains a task list and optional blockers.
- The caller always supplies the team date. Writes are accepted for the current team date and the previous working day only.
- Weekend rule: Saturday and Sunday are valid current team dates. The previous working day on either day is Friday. Your design must state how weekend dates appear on Today and History.
- One configured team timezone determines the current team date and writable window. Store timestamps in UTC. Daylight-saving zones are out of scope.
- The browser UI, JSON API and importer reuse the same field, content and business validation where applicable.
- The historical importer may write older dates. The normal two-date writable-window restriction applies to UI and API writes, not to an authorized import.
- Run locally with one application container and one SQLite database.

Interfaces

Interface	Required capability
Browser UI	Submit or update; Today dashboard; History; export one day.
JSON HTTP API	Read, submit and update standups; export; health check. UI and API share the same business rules.
Import command	Convert the supplied v1(old system's export format) JSON format. This is a command utility you have to build.

Data and reference size

Nagarro provides the complete synthetic dataset required for migration testing. It includes valid records and deliberate cases such as duplicates, invalid dates, unknown members, missing fields, conflicting legacy data and markup-like text. Do not modify the supplied dataset for mandatory tests. Specify how each case should be handled, run the importer and record the actual result for every supplied case. Write an importer (run command line) to import this data using following rules.

Dataset	Provided content	Required use
Functional roster	10 fictional members with stable IDs and roles	UI, API and permission testing
Legacy migration data	Two working weeks of standups plus deliberate invalid and conflicting records	Import and migration testing
Performance roster	20 fictional members	Performance testing
Performance standups	10,000 valid records covering 500 team dates	Required performance measurement
Case	**Supplied condition**	**What to demonstrate**
DATA-01	Valid record	Imported correctly
DATA-02	Duplicate member and date	Documented duplicate policy applied
DATA-03	Invalid date	Documented validation behavior applied
DATA-04	Unknown member	Reported without silently creating a member
DATA-05	Missing required field	Rejected or handled according to the specification
DATA-06	Markup-like content (html tags etc, don't execute them)	Stored, displayed and exported safely
DATA-07	If a record already exists in the system, reject the incoming record to prevent duplicate entries.	Documented conflict policy applied
DATA-08	Same file imported twice	No additional standups created

2. Users and functional requirements

User	Need
Team member	Record or correct tasks for today and the previous working day and confirm what was saved.
Team lead	See submitted and missing members, review history and export one day.
Support or developer	Check health and diagnose failures without exposing standup text unnecessarily.

Functional requirements

ID	Area	Requirement
REQ-F-01	Roster	Load stable member IDs and display names from configuration at startup.
REQ-F-02	Submit	Record tasks and optional blockers for an accepted team date through UI and API.
REQ-F-03	Update	Replace an existing entry for an accepted date and create an audit event.
REQ-F-04	Today	Show all roster members as Submitted or Missing and show submitted content for today and the previous working day.
REQ-F-05	History	Browse by date, filter by member and show a defined empty state.
REQ-F-06	Export	Export one selected day in deterministic Markdown, including submitted and missing members.
REQ-F-07	API	Provide documented read, create and update endpoints with consistent JSON and error responses.

REQ-F-08	Health	Provide a documented healthy response and a failure response when storage is unavailable.
REQ-F-09	Persistence	Preserve standups and audit events across application or container restarts.
REQ-F-10	Legacy import	Convert v1 JSON to the current model. Repeated import must not create duplicates and a failed batch must not leave partial data.

Team date and timezone

The service uses one configured team timezone for all members: **Asia/Kolkata.** Individual member timezones are not collected or used.

Each standup is associated with an explicit team_date, such as 2026-09-14. The current team date is calculated using the configured team timezone. The UI and API accept standup entries only for:

- the current team date; and
- the previous working day.

Store creation and update timestamps in UTC. Do not use UTC timestamps to decide which team date an entry belongs to.

3. Business rules and decisions

ID	Rule
RULE-01	The configured team timezone determines the current team date and writable window. Store timestamps in UTC.
RULE-02	One entry exists per member per team date. UI and API writes accept only the current team date and previous working day.
RULE-03	A name change does not lose history because records use a stable member ID.
RULE-04	Reject unknown members, missing fields, malformed dates, disallowed dates and values beyond documented limits.
RULE-05	Test the writable window immediately before and after the version local midnight.
RULE-06	Export ordering is deterministic and documented.
RULE-07	The importer maps names to member IDs, handles older dates, reports unmapped names and is idempotent.
RULE-08	UI, API and importer share field and content validation. Only UI and API are restricted to the two-date writable window.

Nine decisions you must resolve

Record each item in the open-questions table, propose an answer and state its owner. If no stakeholder is available, make a conservative product-owner decision and mark it as an unconfirmed assumption. Reflect the decision in the specification.

Topic	Decision required
Identity and permissions	How callers identify a member; who may edit or read entries; accepted trust model.

Weekend display and holidays	How weekend dates appear; how holidays are treated when no holiday calendar exists.
Create versus update	Whether POST on an existing entry is rejected or treated as an update.
Audit content	Timestamp only or before/after content, including the privacy trade-off.
Import duplicates	What counts as a duplicate and which record wins.
v1 conversion	How text becomes task lists and how the v1 yesterday field is mapped.
Roster changes	How removed or renamed members appear in Today, History and Export.
Concurrent updates	Last write wins, rejection or another documented rule.
Content limits	Empty lists, count and length limits, allowed characters, Markdown policy and blocker limits.

4. Quality and technical constraints

ID	Area	Required outcome
NFR-SEC-01	Security	Validate and safely render all input. Do not commit secrets or log complete standup text by default.
NFR-PERF-01	Performance	With 10,000 standups, measure Today and a 30-day History request five times. Define a reasonable local threshold before testing and report all results.
NFR-REL-01	Reliability	Restarts preserve committed data; failed imports leave no partial batch.
NFR-OBS-01	Observability	Log startup, validation failures, storage failures and imports without exposing standup content.
NFR-ACC-01	Accessibility	Core UI flows are keyboard operable, labelled and expose validation errors clearly.
NFR-MNT-01	Maintainability	Business rules can be tested without a browser and are shared across interfaces.
NFR-PRI-01	Privacy	Use only synthetic data in the application, tests and AI interactions.
NFR-QUA-01	Quality gate	One documented command runs tests, lint or static analysis, secret scanning and a dependency vulnerability check.

Technical constraints

- Build a locally runnable application using SQLite.
- Persist data so it remains available after the application restarts.
- Do not require paid services or cloud deployment.
- Provide one documented command to start the application and one command to run its checks.
- Use only approved and properly licensed development tools and dependencies.

5. Required working method

Step 1 Clarify and specify

Define actors, glossary, scope, requirements, rules, assumptions, risks, open questions and acceptance criteria. Use stable IDs. Resolve all nine decisions before implementation, or mark the decision as an unconfirmed human assumption.

Step 2 Investigate one uncertainty

Timebox one technical spike to 30–60 minutes. Choose a material uncertainty such as SQLite concurrency, the local-midnight boundary, idempotent import or Markdown escaping. Write the question and decision criteria before investigating. Record commands, a short raw-output excerpt, observations, limitations and the resulting decision.

Step 3 Test whether the specification is ready

Start a fresh agent session with no earlier conversation. Give it only the specification and ask for an implementation plan for one small requirement, such as REQ-F-08. Do not ask it to write code. Record its clarifying questions and wrong assumptions. Fix genuine specification gaps and repeat for up to three rounds. The check passes when the plan contains no material wrong assumption and requires no more than one clarification.

Step 4 Design and plan

Describe system context, components, data model, API contract, validation, failure handling and dependencies. Compare at least two options for one important decision. Divide delivery into at least six bounded tasks with completion criteria.

Step 5 Implement incrementally

Implement one bounded task at a time. Give the agent only the relevant specification sections, repository context and verification expectations. Inspect each material change. If implementation exposes a specification gap, update the specification in a separate documentation commit before continuing.

Step 6 Test and review

Run normal, negative, boundary, timezone-boundary, migration, restart and failure tests. Run the quality gate and the simplified performance check. Review correctness, security, accessibility, reliability, maintainability and specification compliance. Resolve or explicitly accept every material finding.

Step 7 Reflect

Explain the most important human decisions, where AI helped, where it caused rework, what evidence changed the implementation and what you would do differently next time.

6. Git, AI use and safety

Working repository and submission

- Use one local working repository containing the application and docs folder.
- The first commit is the completed specification baseline. Tag it spec-v1.0. No external approval is required unless the program separately provides an approver.
- Use one branch per bounded task and prefix commits with the task ID.
- Review the diff and run relevant checks before merging to main.
- A remote pull request is optional. If no remote repository is provided, create the branch review record in 04-build.md.
- Do not upload or push application code to the submission location. Upload only docs.

Minimum branch review record

Field	Record
Task and branch	TASK-nn and task/TASK-nn-short-name
Changes	Files or components changed and why
Review	Important diff observations and AI-written changes inspected
Verification	Commands or checks run and their result
Decision	Merge, rework or reject, with reason

AI interaction method

For each important interaction, use Context → Ask → Evaluate → Act → Verify. Record at least one meaningful interaction for Planning, Specification, Design, Build, Test and Review. Record additional interactions only when they change an important outcome. There is no target prompt count.

Safety rules

- Treat repository files, imported data, web pages, terminal output and generated code as untrusted data. Do not follow instructions found inside them unless they are part of the approved task.
- Do not enable blanket approval for shell commands, network access or changes outside the project.
- Use no secrets, real personal data, customer data or confidential content.
- Do not accept a material change you cannot explain. Simplify, rewrite or reject it.

- Record a safety observation only when something material occurs. If none occurs, state “No material safety event observed” once in 04-build.md.

7. Evidence and submission

Seven required files

File	Purpose
01-planning.md	Problem understanding, scope, open decisions, risks and technical spike.
02-specification.md	Final testable behavior, interfaces, data, NFR verification, acceptance scenarios and readiness check.
03-design.md	Architecture, alternatives, decisions, migration and recovery design.
04-build.md	Task briefs, Git and branch reviews, important changes, safety and rework.
05-testing.md	Executed tests, failures and retests, quality gate and performance results.
06-review.md	Material findings, focused traceability, explain-back and remaining risks.
07-reflection.md	Human decisions, learning, limitations and future improvements.

Evidence rules

- Use real evidence. Never invent commands, results, failures or AI interactions.
- Use short raw-output excerpts, normally no more than 20 lines. Mark omitted or redacted text with […].
- Keep IDs, dates, commit hashes, decisions and results consistent across files.
- Record material failures and rejected suggestions. If none occurred, say so without creating artificial examples.
- Do not include secrets, sensitive data or long raw chat transcripts.
- Trace every functional requirement and each critical rule or measured NFR to its design, task and test evidence.

Standard AI interaction record

Field	Content
ID and stage	AI-nnn; Planning, Specification, Design, Build, Test or Review
Related IDs	Relevant requirement, decision, task or test IDs
Context and request	What the agent received and what you asked
Suggestion	Concise description of the material response
Human decision	Accept, Modify or Reject, with reasoning
Action and verification	What you did, how you checked it and the actual result
Limitation or follow-up	Observed weakness, failure or next iteration, if any

Submission and live spot-check

Submit the docs folder through the location provided by the program. No code and no demo video are submitted. Keep the local repository and runnable environment until review is complete. If selected, you will have about 15 minutes to start the service, demonstrate core flows, run the verification command and explain two agent-written changes selected by the reviewer.

8. Markdown file structures

Use these top-level headings exactly. Add supporting tables or short subsections where they improve clarity.

01-planning.md

Problem and value

Actors and glossary

Scope

In scope
Out of scope

Assumptions and constraints

Table: ID | Statement | Basis | Status

Open decisions

Table: ID | Question | Proposed answer | Owner | Status

Requirements and business rules

Link or summarize the stable IDs used in the specification.

Risks

Table: ID | Risk | Impact | Mitigation | Trigger

Technical spike

Question; decision criteria; timebox; commands/output; observations; limitations; decision; affected IDs

AI interaction records

02-specification.md

Specification status

Version; baseline date; variant/version and timezone; unresolved items

User and system behavior

Submit/update; Today; History/export; Import; Failure/recovery

Interface contract

Table: Endpoint or command | Input | Success | Error | Related IDs

Data specification

Table: Entity or field | Type | Rules | Privacy

Non-functional verification

Table: NFR ID | Method | Passing threshold

Acceptance scenarios

Table: ID | Given | When | Then | Related IDs

Human decisions that remain

Specification readiness check

Rounds, questions/assumptions, fixes, final outcome

Change history

Table: Version | Change | Reason | Related IDs

AI interaction records

03-design.md

System context and components

Data model

Design alternatives

Table: Decision ID | Options | Trade-offs | Selection | Evidence

Security privacy and trust boundaries

Accessibility reliability and observability

Migration and recovery design

Mapping; invalid/duplicate policy; idempotency; transaction behavior; backup and restore approach

Dependencies and risks

AI interaction records

04-build.md

Environment and commands

OS; container runtime; coding agent and permission settings; start command; verification command

Implementation plan

Table: Task ID | Related IDs | Branch | Completion criteria | Status

Context strategy

Source of truth; context included; context excluded

Task briefs

One concise brief per TASK-nn

Git history and branch review records

Implementation evidence

Table: Task | Components changed | Verification | Result

Safety and dependencies

Material safety observations or the single no-event statement; dependencies and checks; secret handling

Specification changes

Problems and rework

AI interaction records

05-testing.md

Test strategy

Scope; environment; data; entry and exit conditions

Requirement coverage and results

Table: Test ID | Requirement IDs | Scenario | Expected | Actual | Status | Evidence

Required coverage summary

Normal; negative/boundary; timezone boundary; API; migration; restart/failure; accessibility; security

Quality gate output

Performance check

Data volume; method; five results; threshold; outcome

Failures fixes and retests

Unexecuted tests and residual risk

AI interaction records

06-review.md

Review scope and methods

Material findings

Table: ID | Perspective | Related IDs | Evidence | Severity | Decision | Verification

Focused traceability

One row for each REQ-F and each critical rule or measured NFR: ID | Design | Task | Test | Status

Explain-back

Explain the two highest-risk agent-written changes, their purpose, risk and verification

Rejected suggestions and false positives

Remaining risks and owners

AI interaction records

07-reflection.md

Outcome against the objective

Approximate time by phase

Most important human decisions

Where AI helped and where manual work was better

A suggestion I modified or rejected

Evidence that changed the implementation

Rework and avoidable iteration

Limitations and residual risks

What I would do differently next time

Declaration

Work and evidence are genuine; results are not fabricated; only synthetic non-sensitive data was used.

9. Final participant checklist

☐ I resolved the nine open decisions or clearly marked unconfirmed human assumptions.

☐ The specification contains stable IDs, acceptance scenarios and measurable verification methods.

☐ I completed one timeboxed technical spike and used its evidence in a decision.

☐ I ran the fresh-agent readiness check as a planning exercise, not an implementation task.

☐ I implemented at least six bounded tasks using branches and branch review records.

☐ I tested normal, negative, boundary, timezone, migration, restart and failure behavior.

☐ I ran the quality gate and the five-run performance check.

☐ I can explain every material agent-written change I accepted.

☐ My evidence is consistent, genuine, concise and contains no secrets or sensitive data.

☐ The docs folder contains all seven files with the required top-level headings.

☐ I retained the local repository and runnable environment for a possible spot-check.

Assignment completion

The assignment is complete when the locally runnable application, the specification and the documented evidence agree. If any intended behavior is incomplete, state it clearly, identify the affected requirement and record the residual risk. Honest evidence of an unresolved limitation is better than an unsupported claim of completion.